

3D HEIST SIMULATOR

AI Implementation Rule Book & SRS

Goal Stack Planning · STRIPS Operators · A Heuristic Search*

Version 1.0

User = Robber · AI = Police

Normal / Hard Mode

1. PROJECT OVERVIEW

This document is the authoritative implementation reference for the 3D Heist Simulator AI system. It defines every rule, goal, operator, precondition, effect, and heuristic that must be implemented. Feed this entire document to any AI coding assistant before asking it to write or modify code.

1.1 What The Game Is

A turn-based, adversarial 3D heist game rendered with Raylib. The human always plays the Robber. All Police units are AI-controlled. The game runs on a 3D grid (15×15 per floor, 1–3 floors). The AI uses STRIPS-style Goal Stack Planning as its primary decision engine, with A* (using goal-aware heuristics) as the movement executor.

1.2 Core Design Principles

- Every AI decision must trace back to a named Goal on the Goal Stack.
- Every Goal is achieved by a STRIPS Operator with explicit Preconditions and Effects.
- Every movement Operator calls A* with a GoalType-aware heuristic.
- The Debug Window shows the live Goal Stack, active Operator, and A* successor log.
- There is one Debug Window per Police AI agent, spawned at game start.
- The Goal Stack is displayed exactly as solved in class: top-first, with compound goals using '^', completed goals struck through, cancelled goals in red.

1.3 Technology Stack

Component	Technology
Rendering	Raylib (C++)
AI Planning	STRIPS Goal Stack (custom C++)
Pathfinding	A* on 3D grid with GoalType heuristic
Debug UI	Separate Raylib process per Police agent
Agent Comm	Snapshot JSON file per agent (agent_N_snapshot.txt)

2. WORLD & MAP RULES

2.1 Grid Layout

- 15 × 15 cells per floor. Floors numbered 0 (ground) upward.
- Difficulty determines number of floors: Easy=1, Normal=2, Hard=3.
- Coordinates: Position(x, y, z) where z = floor number.

2.2 Cell Types

Cell Type	Symbol	Behaviour
EMPTY	.	Walkable. No special effect.
WALL	#	Impassable. Cannot be entered by any agent.
VAULT	V	Robber collects vault by occupying this cell.
EXIT	E	Robber escapes by occupying this cell after collecting vault.
STAIRS_UP	U	Stepping on this cell moves agent up one floor automatically.
ELEVATOR_DN	D	Stepping on this cell moves agent down one floor automatically.
CCTV_ZONE	C	Increases police heuristic awareness. Costs +2 for robber path.
ALERT_ZONE	A	Triggers police boost (2 moves/turn for N turns). Consumed on trigger.

2.3 Floor Transition Rules

- Robber: may use STAIRS_UP / ELEVATOR_DN freely at any time.
- Police: may only use floor transitions if (vaultStolen == true) OR (alertBoostActive == true).
- Floor transition is automatic — stepping on a transition cell instantly moves agent to next floor.

■■ Police on Floor 0 cannot chase Robber to Floor 1 until one of the two conditions above is met.

2.4 Alert Zone Mechanics

- When Robber steps on ALERT_ZONE: alertTriggered = true, cell converts to CCTV_ZONE (consumed).
- Police boost activates: policeBoostTurnsRemaining = 5 (difficulty-scaled). During boost police move 2 steps/turn.
- RuleEngine.notifyAlertTriggered(pos) is called. The alertPos is stored for planner use.
- When police reaches alertPos, respond(Police, Alert) goal is marked complete. Police then push protect(Police, Vault) back onto stack.

3. AGENTS

3.1 Robber (Human Controlled)

- Controlled via WASD keys. One move per turn.
- Objective: collect vault, then reach exit.
- Can step on ALERT_ZONE to distract police (intentional tactic).
- Floor transition: automatic on stepping onto U/D cell.
- Win condition: Robber.pos == Exit.pos AND Robber.hasVault == true.

3.2 Police (AI Controlled — Multiple Agents)

- One Police agent per 'P' spawn marker in the map. Typically 1–3 agents.
- Each Police agent has its own independent GoalStack and PoliceAIPlanner instance.
- Each Police agent has its own Debug Window (separate process).
- Police are identified as Police1, Police2, Police3, etc. (1-indexed).
- Police agents do NOT block each other — they may occupy different cells.
- Lose condition: any Police.pos == Robber.pos.

3.3 Police Activation Rules (CRITICAL)

Police activation is floor-and-situation based. This is the most important behavioural rule:

Situation	Which Police Activate	Goal Pushed
Game starts	None	All push stayHalt(PoliceN)
Robber makes first move on Floor 0 (Vault NOT on Floor 0)	Police on Floor 0 only	protect(Police1, Upstairs)
Robber makes first move on Floor 0 (Vault IS on Floor 0)	Police on Floor 0 only	protect(Police1, Vault)
Robber reaches Floor 1 (Vault on Floor 1)	Police on Floor 1 only	protect(Police2, Vault)
Alert zone triggered (any floor)	Police on SAME FLOOR as alert	respond(PoliceN, Alert)
Vault stolen	ALL police, all floors	catch(PoliceN, Robber) — each independently

■■ Police on Floor 0 do NOT follow Robber to Floor 1 unless vaultStolen OR alertBoostActive. When Robber moves between floors, each Police stays on its own floor and independently manages its own goal stack.

4. GOAL STACK SPECIFICATION

The Goal Stack is the primary AI decision mechanism. It follows the exact classroom STRIPS Goal Stack Planning model: goals and operators are pushed and popped from a single stack. The top of the stack is always the current focus. Compound goals use '^'. Operators are executable actions. Goals are conditions to be satisfied.

4.1 GoalEntry Structure

```
struct GoalEntry {  
    string goalExpression; // exact string – see Section 5 for all valid strings  
    bool isOperator; // true = executable action, false = goal/precondition  
    string operatorName; // 'MOVE', 'STEAL', 'ESCAPE', 'CAPTURE', etc.  
    vector preconditions; // human-readable strings for debug display  
    vector effects; // human-readable strings for debug display  
    enum Status { PENDING, ACTIVE, PERFORMING, COMPLETED, CANCELLED } status;  
};
```

4.2 Stack Display Rules (Debug Window)

- ACTIVE → Yellow text. The current focus of the AI.
- PENDING → White text. Waiting below the active entry.
- PERFORMING → Green text. Operator is mid-execution (A* running across turns).
- COMPLETED → Green text + strikethrough line drawn through text.
- CANCELLED → Red text. Goal was interrupted.
- Compound goals (contain '^') → Purple text.
- Top-level/root goal (bottommost non-operator) → Red text.

4.3 GoalStack Operations

Method	Behaviour
push(entry)	Demotes current ACTIVE to PENDING. Pushes new entry as ACTIVE (top = back of vector).
pop()	Removes top entry. Promotes next non-completed entry to ACTIVE.
peek()	Returns top entry without removing. top = back() of internal vector.
markComplete()	Sets top entry status to COMPLETED. Adds to completedGoals list.
markPerforming()	Sets top entry status to PERFORMING (operator running across turns).
finalizeCompleted()	Pops all COMPLETED/CANCELLED entries from top. Promotes next ACTIVE.
cancel(expr)	Finds matching expression, marks CANCELLED, adds to cancelledGoals.
cancelAll()	Cancels entire stack. Used when vault stolen mid-other-goal.
getStack()	Returns entries top-first (index 0 = top). Used by snapshot writer.

5. EXACT GOAL EXPRESSION STRINGS

These are the ONLY valid goalExpression strings. Use them verbatim everywhere — in GoalEntry construction, snapshot JSON, string comparisons, and debug display. Any deviation will break the planner logic.

5.1 Police Goal Expressions

Situation	goalExpression string	isOp
Initial / waiting	"stayHalt(PoliceN)"	false
Protect when vault on floor	"protect(PoliceN, Vault)"	false
Protect precondition	"sameFloor(PoliceN, Vault) ^ pathAvailable(PoliceN, Vault)"	false
Protect operator	"MOVE(PoliceN, Vault)"	true
Protect when vault NOT on floor	"protect(PoliceN, Upstairs)"	false
Floor move precond	"sameFloor(PoliceN, Upstairs) ^ pathAvailable(PoliceN, Upstairs)"	false
Floor move operator (up)	"MOVE(PoliceN, Upstairs)"	true
Floor move operator (down)	"MOVE(PoliceN, Downstairs)"	true
Floor move precond	"onTransitionCell(PoliceN)"	false
Alert response goal	"respond(PoliceN, Alert)"	false
Alert precondition	"sameFloor(PoliceN, Alert) ^ pathAvailable(PoliceN, Alert)"	false
Alert operator	"MOVE(PoliceN, Alert)"	true
Chase goal	"catch(PoliceN, Robber)"	false
Chase precondition	"sameFloor(PoliceN, Robber) ^ pathAvailable(PoliceN, Robber)"	false
Chase operator	"MOVE(PoliceN, Robber)"	true
Capture operator	"CAPTURE(PoliceN, Robber)"	true

■■ Replace N with the 1-based police index: Police1, Police2, Police3. This makes each agent's stack uniquely identifiable in the debug window.

5.2 Robber Goal Expressions (AI Robber mode only)

Situation	goalExpression string	isOp
Root compound goal	"steal(Robber, Vault) ^ escape(Robber, Exit)"	false
Steal sub-goal	"steal(Robber, Vault)"	false
Steal precondition	"atVault(Robber) ^ vaultClear(Vault)"	false
Steal operator	"STEAL(Robber, Vault)"	true

Move to vault operator	"MOVE(Robber, Vault)"	true
Vault move precondition	"sameFloor(Robber, Vault) ^ pathAvailable(Robber, Vault)"	false
Distract goal	"distract(Robber, Alert)"	false
Distract precondition	"sameFloor(Robber, AlertZone) ^ pathAvailable(Robber, AlertZone)"	false
Distract operator	"MOVE(Robber, AlertZone)"	true
Escape sub-goal	"escape(Robber, Exit)"	false
Escape precondition	"atExit(Robber) ^ hasVault(Robber)"	false
Escape operator	"ESCAPE(Robber)"	true
Move to exit operator	"MOVE(Robber, Exit)"	true
Exit move precondition	"sameFloor(Robber, Exit) ^ pathAvailable(Robber, Exit)"	false
Floor up operator	"MOVE(Robber, Upstairs)"	true
Floor down operator	"MOVE(Robber, Downstairs)"	true
Evade goal	"evade(Robber, Police)"	false
Evade precondition	"pathAvailable(Robber, SafeCell) ^ maxDistPolice(Robber)"	false
Evade operator	"MOVE(Robber, SafeCell)"	true

6. POLICE GOAL STACK — FULL WALKTHROUGH

6.1 Game Start

All Police push stayHalt. Stack is identical for every Police agent at start:

Police1 STACK at game start

```
stayHalt(Police1) [ACTIVE] ← top
```

6.2 Robber Makes First Move — Vault NOT On This Floor

Robber moves on Floor 0. Vault is on Floor 1. Police1 is on Floor 0.

Police1 detects: robberMadeFirstMoveOnFloor[0] = true. Vault.floor != Police.floor.

stayHalt is marked COMPLETE. protect(Police1, Upstairs) block is pushed:

Police1 STACK — Robber on Floor 0, Vault on Floor 1
MOVE(Police1, Upstairs) [ACTIVE] ← top
sameFloor(Police1, Upstairs) ^ pathAvailable(Police1, Upstairs) [PENDING]
protect(Police1, Upstairs) [PENDING]

MOVE(Police1, Upstairs) executes: A* finds nearest STAIRS_UP cell. Police1 moves toward it. Once on transition cell, floor changes. Goal marks COMPLETE.

6.3 Robber Makes First Move — Vault ON This Floor

Robber moves on Floor 1. Vault is also on Floor 1. Police2 is on Floor 1.

Police2 detects: robberMadeFirstMoveOnFloor[1] = true. Vault.floor == Police.floor.

stayHalt is marked COMPLETE. protect(Police2, Vault) block is pushed:

Police2 STACK — Robber on Floor 1, Vault on Floor 1
MOVE(Police2, Vault) [ACTIVE] ← top
sameFloor(Police2, Vault) ^ pathAvailable(Police2, Vault) [PENDING]
protect(Police2, Vault) [PENDING]

6.4 Alert Zone Triggered (Police mid-protect)

Robber steps on ALERT_ZONE on Floor 1. respond block is pushed ON TOP without cancelling protect:

```

    Police2 STACK — Alert triggered during protect
    MOVE(Police2, Alert) [ACTIVE] ← top
    sameFloor(Police2, Alert) ^ pathAvailable(Police2, Alert) [PENDING]
    respond(Police2, Alert) [PENDING]
    ■■ protect block below (intact, not cancelled)
    MOVE(Police2, Vault) [PENDING]
  
```


<code>sameFloor(Police2, Vault) ^ pathAvailable(Police2, Vault) [PENDING]</code>
<code>protect(Police2, Vault) [PENDING]</code>

Police2 moves toward Alert. When `Police2.pos == alertPos`:

- `respond(Police2, Alert)` marked COMPLETE.
- `respond` block is finalized (popped).
- `protect(Police2, Vault)` block resumes as ACTIVE — police returns to protecting vault.

6.5 Vault Stolen Mid-Alert-Response (INTERRUPT — Critical)

Police2 is moving toward Alert (MOVE(Police2, Alert) is PERFORMING). Robber reaches vault. vaultJustStolen = true.

Interrupt fires at TOP of runTurn(), BEFORE executeTop():

Step 1: cancelAll() — entire stack is cancelled:

Police2 STACK — immediately after cancelAll()
MOVE(Police2, Alert) [CANCELLED]
sameFloor ^ pathAvail... [CANCELLED]
respond(Police2, Alert) [CANCELLED]
MOVE(Police2, Vault) [CANCELLED]
sameFloor ^ pathAvail... [CANCELLED]
protect(Police2, Vault) [CANCELLED]

Step 2: pushCatchBlock(world) — catch block pushed:

Police2 STACK — after vault stolen interrupt
MOVE(Police2, Robber) [ACTIVE] ← top
sameFloor(Police2, Robber) ^ pathAvailable(Police2, Robber) [PENDING]
catch(Police2, Robber) [PENDING]

■■ Stack insertion order for catch block — push in this exact sequence so top is MOVE: (1) push catch(Police2, Robber) (2) push sameFloor ^ pathAvailable compound (3) push MOVE(Police2, Robber) ← this becomes top

6.6 Vault Stolen — Floor Transition Needed (Multi-Floor Chase)

Police1 is on Floor 0. Robber is on Floor 1. Vault stolen. Police1 needs to chase.

Police1 STACK — vault stolen, Robber on different floor
MOVE(Police1, Upstairs) [ACTIVE] ← top
onTransitionCell(Police1) [PENDING]
sameFloor(Police1, Robber) ^ pathAvailable(Police1, Robber) [PENDING]
MOVE(Police1, Robber) [PENDING]
catch(Police1, Robber) [PENDING]

Stack insertion order when floors differ — push in this sequence:

- (1) push catch(Police1, Robber)
- (2) push sameFloor(Police1, Robber) ^ pathAvailable(Police1, Robber)
- (3) push MOVE(Police1, Robber)
- (4) push onTransitionCell(Police1) // floor precondition
- (5) push MOVE(Police1, Upstairs/Downstairs) // this is now top

■■ MOVE(Police1, Robber) must be RE-PLANNED every turn since Robber moves. When MOVE(Police1, Robber) becomes ACTIVE again, call A* fresh with current Robber pos.

6.7 Police2 Chase — Same Floor (Simpler Stack)

Police2 is on Floor 1. Robber is also on Floor 1. Vault stolen.

sameFloor precondition is already satisfied — no floor move needed:

Police2 STACK — vault stolen, same floor as Robber
MOVE(Police2, Robber) [ACTIVE] ← top
sameFloor(Police2, Robber) ^ pathAvailable(Police2, Robber) [PENDING]
catch(Police2, Robber) [PENDING]

7. STRIPS OPERATORS — COMPLETE DEFINITION

Every operator has: Name, Preconditions (must be true before execution), Effects (what changes after execution), and Execution method.

■ MOVE(Agent, Target)

Preconditions	Effects	Execution
• pathExists(Agent.pos, Target) • NOT wall(Target) • Agent.pos is valid	→ Agent.pos = next step on A* path to Target	Call A* (Agent.pos, Target) with GoalType heuristic. Execute first step.

■ MOVE_FLOOR(Agent, TargetFloor) — via MOVE to Upstairs/Downstairs

Preconditions	Effects	Execution
• transitionCellExists on Agent.currentFloor • IF Agent==Police: vaultStolen==true OR alertBoostActive==true	→ Agent.floor = TargetFloor (automatically stepping UP or DOWN only)	Find one step to STAY or call ELEVATOR_DN on current floor. Call MOVE(Agent, TargetPos) with GoalType heuristic.

■ STEAL(Robber, Vault)

Preconditions	Effects	Execution
• Robber.pos == Vault.pos • vaultStolen == false	→ vaultStolen = true → Robber.hasVault = true	Direct state update. No A* needed. Triggers vaultJustStolen flag for Police.

■ ESCAPE(Robber)

Preconditions	Effects	Execution
• Robber.pos == Exit.pos • Robber.hasVault == true	→ robberWon = true → gameOver = true	Direct state update. Triggers game-over ROBBER_WON.

■ CAPTURE(PoliceN, Robber)

Preconditions	Effects	Execution
• Police.pos == Robber.pos	→ policeWon = true → gameOver = true	Direct state update. Triggers game-over POLICE_WON.

■ DISTRACT(Robber, AlertZone)

Preconditions	Effects	Execution
• alertZoneExists on Robber.currentFloor • NOT Robber.pos == AlertZonePos	→ alertTriggered = true (when stepped on)	Robber intentionally steps on ALERT_ZONE. Indirect — triggered by Robber.

■ STAY_HALT(PoliceN)

Preconditions	Effects	Execution
• always true	→ no change	Police does nothing. Returns current pos. Stack entry stays ACTIVE.

8. INTERRUPT PRIORITY SYSTEM

Interrupts are checked at the VERY START of every Police agent's `runTurn()`, before any goal decomposition or operator execution. They override whatever the police was doing. Order of priority:

Priority	Trigger Condition	Action	Stack Operation
1 (HIGH EST)	<code>vaultJustStolen == true AND catch not already on stack</code>	Cancel everything. Chase robber.	<code>cancelAll()</code> <code>pushCatchBlock(world)</code>
2	<code>alertJustTriggered == true AND respond not already on stack AND police on same floor as alert</code>	Push respond on top. Keep protect below.	<code>pushRespondBlock(world)</code> (no cancel)
3 (LOW EST)	<code>robberMadeFirstMoveOnFloor AND stayHalt is active</code>	Replace <code>stayHalt</code> with appropriate protect goal.	<code>markComplete(stayHalt)</code> <code>pushProtectBlock(world)</code>

■■ `vaultJustStolen = (world.vaultStolen == true AND lastVaultStolen == false)`. `alertJustTriggered = (world.alertTriggered == true AND lastAlertTriggered == false)`. Always update `lastVaultStolen` and `lastAlertTriggered` at end of `handleInterrupts()`.

8.1 runTurn() Execution Order

Every Police agent's `runTurn()` must follow this exact sequence every turn:

Step	Action	Code
1	Initialize if first turn	<code>ensureInitialStack(world)</code>
2	Clean up completed/cancelled from top	<code>goalStack.finalizeCompleted()</code>
3	Check interrupts	<code>handleInterrupts(world)</code>
4	Execute top of stack (ONE move)	<code>executeTop(world)</code>
5	Return new position	<code>return world.policePos</code>

9. HEURISTIC ENGINE — A* GOAL-AWARE HEURISTICS

A* is called by planners to execute MOVE operators. The heuristic function takes a GoalType parameter so it optimises for the current goal, not a generic distance. All heuristics use Manhattan distance as the base. Difficulty adjusts weights.

9.1 GoalType Enum

```
enum GoalType {
    NONE, PROTECT_VAULT, CHASE_ROBBER, RESPOND_ALERT,
    REACH_VAULT, ESCAPE_TO_EXIT, REACH_ALERT_ZONE, MOVE_FLOOR_GOAL
};
```

9.2 Heuristic Formulas

GoalType	Formula	Used by
PROTECT_VAULT	$w1 * \text{manhattan}(n, \text{VaultPos}) + w2 * (1 / (\text{distToRobber} + 1)) + w3 * \text{cctvBonus}(n) + w4 * \text{alertZonePenalty}(n)$	Police — protect/respond-after
CHASE_ROBBER	$w1 * \text{manhattan}(n, \text{RobberPos}) + w2 * \text{floorDiffPenalty}(n, \text{Robber}) + w3 * \text{transitionBonus}(n)$	Police — catch
RESPOND_ALERT	$w1 * \text{manhattan}(n, \text{AlertPos}) + w2 * \text{vaultExposurePenalty}(n, \text{VaultPos})$	Police — respond
MOVE_FLOOR_GOAL	$w1 * \text{manhattan}(n, \text{nearestTransitionCell}) + w2 * \text{floorDiffPenalty}$	Police/Robber — floor change
REACH_VAULT	$w1 * \text{manhattan}(n, \text{VaultPos}) + w2 * \text{policeDanger}(n) + w3 * \text{alertZoneBonus}(n)$	Robber — steal phase
ESCAPE_TO_EXIT	$w1 * \text{manhattan}(n, \text{ExitPos}) + w2 * \text{policeDanger}(n) + w3 * \text{transitionBonus}(n)$	Robber — escape phase
REACH_ALERT_ZONE	$w1 * \text{manhattan}(n, \text{nearestAlertZone}) + w2 * \text{policeDanger}(n)$	Robber — distract

9.3 Weight Scaling by Difficulty

Difficulty	w1 (distance)	w2 (danger/proximity)	Behaviour
Easy	1.0	0.3	Police less aggressive, forgives mistakes
Normal	1.0	0.7	Balanced pursuit and protection
Hard	1.0	1.5	Police very aggressive, blocks paths

10. DEBUG WINDOW — SPECIFICATION

10.1 One Window Per Police Agent

At game start, GameEngineGUI spawns one debug_window process per Police agent:

```
// For N police agents:
for (int i = 0; i < policeList.size(); i++) {
    string snapshotFile = "debug_snapshot_police" + to_string(i+1) + ".txt";
    pid_t pid = fork();
    if (pid == 0) {
        string title = "Police" + to_string(i+1) + " Debug";
        execl("./debug_window", "./debug_window",
            snapshotFile.c_str(), title.c_str(), nullptr);
        _exit(1);
    }
}
```

Each Police agent's PoliceAIPlanner writes to its own snapshot file after every turn. The debug window reads its assigned file continuously (30fps refresh).

10.2 Window Layout — Three Panels

Window minimum size: 1100 × 750. Font: monospace (Courier/Menlo). Three equal-width panels side by side, full height.

Panel 1 — Goal Stack	Panel 2 — Active Operator	Panel 3 — A* Successors
Header: GOAL STACK Agent: PoliceN GoalType: PROTECT_VAULT ■■■■■■■■■■■■■■■■■■■■ [ACTIVE] MOVE(Police1, Vault) ← yellow [PENDING] sameFloor ^ path... [PENDING] protect(Police1, Vault) ■■■■■■■■■■■■■■■■■■■■ COMPLETED ~~stayHalt(Police1)~~ CANCELLED respond(Police1, Alert)	Header: ACTIVE OPERATOR Current Cell: [x,y,z] Target Cell: [x,y,z] ■■■■■■■■■■■■■■■■■■■■ Operator: MOVE ■■■■■■■■■■■■■■■■■■■■ Preconditions: ✓ sameFloor(Police1, Vault) ✓ pathAvailable(Police1, Vault) Effects: → Police1.pos = Vault.pos	Header: A* SUCCESSORS GoalType: PROTECT_VAULT ■■■■■■■■■■■■■■■■■■■■ Pos g h f * ■■■■■■■■■■■■■■■■■■■■ [7,6,0] 1.0 4.2 5.2 [7,7,0] 1.0 3.0 4.0 ★ ← yellow row [8,7,0] 1.0 3.8 4.8 ■■■■■■■■■■■■■■■■■■■■ Chosen Node Pos:[7,7,0] g=1.0 h=3.0 f=4.0

10.3 Debug Window Title

Each window title must include the agent name: 'Police1 — Debug Window', 'Police2 — Debug Window', etc. This is set via the second execl argument and used in the Raylib window title.

11. SNAPSHOT FILE FORMAT

Each Police agent writes to its own snapshot file. Format is JSON-like structured text. Written atomically (write to .tmp then rename to avoid partial reads).

```
{
  "agent": "Police1",
  "state": "PROTECT_VAULT",
  "currentCell": [7, 6, 0],
  "targetCell": [7, 7, 0],
  "goalStack": [
    {
      "expression": "MOVE(Police1, Vault)",
      "isOperator": true,
      "operatorName": "MOVE",
      "status": "ACTIVE",
      "preconditions": ["sameFloor(Police1, Vault)", "pathAvailable(Police1, Vault)"],
      "effects": ["Police1.pos=Vault.pos"]
    },
    {
      "expression": "sameFloor(Police1, Vault) ^ pathAvailable(Police1, Vault)",
      "isOperator": false,
      "status": "PENDING",
      "preconditions": [], "effects": []
    },
    { "expression": "protect(Police1, Vault)", "isOperator": false, "status": "PENDING", ... }
  ],
  "completedGoals": [
    { "expression": "stayHalt(Police1)", "status": "COMPLETED", ... }
  ],
  "cancelledGoals": [],
  "astarSuccessors": [
    { "label": "[7,6,0]", "pos": [7,6,0], "g": 1, "h": 4.2, "f": 5.2, "chosen": false },
    { "label": "[7,7,0]", "pos": [7,7,0], "g": 1, "h": 3.0, "f": 4.0, "chosen": true }
  ],
  "chosenNode": { "label": "[7,7,0]", "pos": [7,7,0], "g": 1, "h": 3.0, "f": 4.0 },
  "activeGoalType": "PROTECT_VAULT"
}
```

■ ■ *goalStack array is ordered top-first (index 0 = top of stack). The debug window must display it top-first.*
astarSuccessors contains only cells adjacent to the search start (4-directional filter).

12. GAME ENGINE TURN LOOP (GameEngineGUI)

12.1 Human-Robber Turn Sequence

Step	Action
1	Read WASD input → compute targetPos.
2	Validate move: not wall, not out of bounds, delta==1.
3	setPlayerPosition(targetPos) → auto floor transition if on STAIRS/ELEVATOR.
4	refreshVaultCollectionState(): if Robber.pos == VaultPos → set vaultStolen, call rules.notifyVaultStolen().
5	Check if Robber stepped on ALERT_ZONE: call rules.notifyAlertTriggered(pos), set alertTriggered=true.
6	updateAI(): run every Police agent's planner. Each planner gets updated WorldState.
7	checkWinConditions(): any Police.pos==Robber.pos → POLICE_WON. Robber.pos==Exit+hasVault → ROBBER_WON.
8	Write snapshot for each Police agent (separate files).
9	Render frame.
10	turnCount++.

12.2 WorldState Fields Required by Planners

```
struct WorldState {  
    const Grid3D* grid;  
    Position policePos; // this police agent's position  
    Position robberPos;  
    Position vaultPos;  
    Position exitPos;  
    Position alertPos; // from RuleEngine.getAlertPos()  
    bool vaultStolen; // from RuleEngine.isVaultStolen()  
    bool robberHasVault;  
    bool alertTriggered; // from RuleEngine.isAlertActive()  
    bool boostActive; // from RuleEngine.isBoostActive()  
    bool policeWon;  
    bool robberWon;  
    vector policePositions; // all police positions  
    GoalType activeGoalType;  
    Position activeTargetPos;  
    int activeTargetFloor;  
};
```

13. MULTI-POLICE ARCHITECTURE

13.1 Per-Agent Independence

- Each PoliceAI has its own PoliceAIPlanner instance.
- Each PoliceAIPlanner has its own GoalStack, lastVaultStolen, lastAlertTriggered, robberMadeFirstMoveOnFloor.
- Each planner writes to its own snapshot file: debug_snapshot_police1.txt, debug_snapshot_police2.txt, etc.
- Each planner's runTurn() receives a WorldState with that agent's policePos.
- Police agents do not share or read each other's goal stacks.

13.2 Floor-Based Activation Tracking

Each PoliceAIPlanner must track per-floor first-move detection independently:

```
// In PoliceAIPlanner:
bool robberMadeFirstMoveOnFloor[3] = {false, false, false}; // indexed by floor 0-2

// In handleInterrupts():
int robberFloor = world.robberPos.z;
if (!robberMadeFirstMoveOnFloor[robberFloor] && world.robberPos != prevRobberPos) {
    robberMadeFirstMoveOnFloor[robberFloor] = true;
    if (world.policePos.z == robberFloor) { // this police is on same floor
        // activate this police
        if (world.vaultPos.z == robberFloor) {
            pushProtectVaultBlock(world); // protect(PoliceN, Vault)
        } else {
            pushProtectUpstairsBlock(world); // protect(PoliceN, Upstairs)
        }
    }
}
```

13.3 Police Stay On Their Floor

A Police agent on Floor 0 must NOT autonomously switch to Floor 1 when Robber moves there, unless vaultStolen OR alertBoostActive. The protect(Police1, Upstairs) goal moves police toward the transition cell, but the planner should NOT push further floor goals until either condition is met.

■ ■ *Exception: If Police1 is already mid-path to Upstairs via protect(Police1, Upstairs) and vault is stolen, the interrupt fires, cancels that goal, and pushes catch(Police1, Robber) which now includes the floor transition logic.*

13.4 Alert Response — Floor Restriction

respond(PoliceN, Alert) is only pushed for Police agents on the SAME FLOOR as the alert zone. Police on other floors ignore alert triggers unless vault is stolen (which triggers catch instead).

14. RULE ENGINE REQUIREMENTS

RuleEngine must expose these methods cleanly for planners to use:

Method	Returns	Description
isVaultStolen()	bool	True after Robber collected vault.
isAlertActive()	bool	True while boost turns remaining > 0.
isBoostActive()	bool	Same as isAlertActive(). Police can use stairs.
getAlertPos()	Position	Last triggered alert zone position.
getAlertFloor()	int	Floor of last alert zone.
notifyVaultStolen()	void	Called by GameEngineGUI when vault collected.
notifyAlertTriggered(pos)	void	Called when Robber steps on ALERT_ZONE.
getPredictionDepth()	int	Depth for PredictionEngine (difficulty-scaled).
getWeights(GoalType)	HeuristicWeights	Difficulty-tuned weights for each goal type.

14.1 HeuristicWeights Struct

```
struct HeuristicWeights {  
    float w1; // distance-to-goal weight  
    float w2; // danger/proximity weight  
    float w3; // bonus/penalty weight (CCTV, alert, transition)  
    float w4; // secondary bonus weight  
};
```

15. DO NOT CHANGE — PROTECTED COMPONENTS

These components must NOT be modified when implementing the AI changes. They are stable and correct.

File / Component	Why Protected
Grid3D.h / Grid3D.cpp	Cell types, floor layout, map loading, getVaultPos/getExitPos — all correct.
RaylibRenderer.cpp	Visual rendering style, tile colors, agent markers — keep as-is.
Win/loss condition checks	In GameEngineGUI.cpp checkWinConditions() — conditions are correct.
debug_main.cpp structure	Separate process architecture is correct. Only update snapshot schema reading.
RuleEngine alert/boost tracking	Internal boost counter logic is correct. Only add new public methods.
AStar3D findPath core	Core A* algorithm is correct. Only add GoalType parameter passing.

15.1 Implementation Order

When handing this document to an AI coding assistant, instruct it to implement in this order:

Order	File	Changes
1	RuleEngine.h/.cpp	Add getAlertFloor(), getWeights(), HeuristicWeights struct.
2	GoalStack.h/.cpp	Add cancelAll(), fix getStack() top-first ordering, fix promoteTopToActive().
3	HeuristicEngine.h/.cpp	Replace monolithic heuristic with compute(node, GoalType, WorldState).
4	AStar3D.cpp	Add GoalType parameter, pass to HeuristicEngine::compute().
5	PoliceAIPlanner.h/.cpp	Full redesign per Section 6 and 8. Per-floor activation. Interrupt system.
6	RobberAIPlanner.h/.cpp	Redesign goal stack per Section 5.2.
7	PoliceAI.cpp	Thin wrapper. Delegates all to PoliceAIPlanner.
8	RobberAI.cpp	Thin wrapper. Delegates all to RobberAIPlanner.
9	GameEngineGUI.cpp	Multi-snapshot writer (one per police). Multi-window spawner. Turn loop update.
10	DebugWindow.cpp/.h	Three-panel layout. Title includes agent name. Accept agent title arg.

16. QUICK REFERENCE — POLICE GOAL PUSH ORDER

This section shows the EXACT push sequence for every scenario. Remember: the LAST item pushed becomes the TOP of the stack.

protect(PoliceN, Vault) Block — push in this order:

```
goalStack.push( makeGoal('protect(PoliceN, Vault)', {...}, {...}) ); // 1st pushed → bottom
goalStack.push( makeGoal('sameFloor(PoliceN, Vault) ^ pathAvailable(...)', {}, {}) ); // 2nd
// IF different floor:
goalStack.push( makeOp('MOVE(PoliceN, Upstairs)', 'MOVE', {...}, {...}) ); // 3rd if needed
goalStack.push( makeOp('MOVE(PoliceN, Vault)', 'MOVE', {...}, {...}) ); // LAST → top
```

respond(PoliceN, Alert) Block — push in this order:

```
goalStack.push( makeGoal('respond(PoliceN, Alert)', {...}, {...}) ); // 1st pushed → bottom of block
goalStack.push( makeGoal('sameFloor(PoliceN, Alert) ^ pathAvailable(...)', {}, {}) ); // 2nd
// IF different floor:
goalStack.push( makeOp('MOVE(PoliceN, Upstairs)', 'MOVE', {...}, {...}) ); // 3rd if needed
goalStack.push( makeOp('MOVE(PoliceN, Alert)', 'MOVE', {...}, {...}) ); // LAST → top
```

catch(PoliceN, Robber) Block — push in this order:

```
goalStack.push( makeGoal('catch(PoliceN, Robber)', {...}, {...}) ); // 1st pushed → bottom
goalStack.push( makeGoal('sameFloor(PoliceN, Robber) ^ pathAvailable(...)', {}, {}) ); // 2nd
// IF different floor:
goalStack.push( makeOp('onTransitionCell(PoliceN)', ...) ); // 3rd if needed
goalStack.push( makeOp('MOVE(PoliceN, Upstairs/Downstairs)', 'MOVE', {...}, {...}) ); // 4th
goalStack.push( makeOp('MOVE(PoliceN, Robber)', 'MOVE', {...}, {...}) ); // LAST → top
```

■ ■ The LAST push is ALWAYS the immediate executable action (a MOVE operator). This ensures `executeTop()` always finds an operator to execute first, then works down as goals complete.

Alert → Protect Resume Rule

When `respond(PoliceN, Alert)` is COMPLETED (police reached `alertPos`):

```
goalStack.markComplete(); // completes respond
goalStack.finalizeCompleted(); // pops respond block
// Now protect block below is exposed – push it again fresh:
pushProtectVaultBlock(world); // re-push protect(PoliceN, Vault)
```

■ ■ Do NOT assume the old protect block survived below respond. Always re-push protect after alert is resolved.

17. COMMON MISTAKES TO AVOID

Mistake	Consequence	Correct Behaviour
Push MOVE before pushing the goal it serves	Stack is inverted — goal is on top, operator never executes	Push goal FIRST (goes to bottom), push operator LAST (becomes top).
Checking <code>goalStack.getStack()[0]</code> as bottom	Display and logic are inverted	<code>getStack()</code> returns top-first. Index 0 = top. <code>back()</code> of internal vector = top.
Using same snapshot file for all police	Debug windows show same agent, multi-agent invisible	One snapshot file per police: <code>debug_snapshot_police1.txt</code> etc.
Pushing catch without <code>cancelAll()</code> first	Old protect/respond goals pile up below catch	Always <code>cancelAll()</code> before <code>pushCatchBlock()</code> on vault stolen.
Not re-planning <code>MOVE(Police, Robber)</code> each turn	Police chases old robber position, never catches moving target	<code>MOVE(Police, Robber)</code> calls A* fresh with current <code>world.robberPos</code> every turn it is ACTIVE.
Police crossing floors without vault stolen	Breaks game rule, police cheats	Only push <code>MOVE(Police, Upstairs)</code> if <code>vaultStolen</code> OR <code>alertBoostActive</code> .
Alert response for police on wrong floor	Police across floors chases alert it cannot reach	Only push respond block for police whose <code>policePos.z == alertPos.z</code> .
Empty <code>goalExpression</code> strings	Debug window shows blank boxes	EVERY <code>GoalEntry</code> push must set <code>goalExpression</code> to a non-empty string from Section 5.
Not writing A* successor log to snapshot	A* panel shows 'No search data'	After every A* call: <code>lastAstarTrace = AStar3D::getLastSearchTrace();</code> write to snapshot.